

Работа с Кодирующими Агентами

Автоматический Просмотр кода

 Copy page 

Автоматический ревью-код для Claude Code с использованием хуков и OpenRouter — асинхронные, неблокирующие проверки от второй модели, пока вы продолжаете работать

 Эталонная реализация доступна по адресу [redline](#). Ключевые особенности:

- Клод (или вы) решает, когда необходима проверка.
- Код Клода полностью контролируется агентом рецензента — он выполняется как асинхронный фоновый процесс.
- Оба агента доступны для наблюдения, настройки и мониторинга затрат в [OpenRouter](#)
- Войдите в оба агента с помощью одной команды: `redline login`
- Реализован как единый стоп-хук кода Claude, обеспечивающий прозрачность и настраиваемость используемого агента и моделей.

Чего это позволяет добиться

Каждый раз, когда Claude Code завершает ответ и в коде остаются незафиксированные изменения, хук автоматически запускает фоновую проверку кода Codex, не прерывая ваш рабочий процесс. Вы продолжаете работать, пока выполняется проверка. Когда она заканчивается, Claude считывает результаты и представляет их.



Claude Code Stop event

```
2   → redline check                (fast, <1s)
3   → git diff --stat HEAD          (any uncommitted changes?)
4   → hash diff stat, compare to .git/redline-last-diff
5   → if changes exist AND diff has changed since last check:
6       save hash to .git/redline-last-diff
7       output { "decision": "block", "reason": "..." }
8       reason includes diff stat summary
9       Claude decides if changes warrant a review
10      Claude spawns 'redline review' as background task
11      → codex exec review streams output in real-time
12      → user can monitor, kill, or keep working
13      → when done, Claude reads output, presents findings
14  → if no changes OR same diff as last check:
15      exit 0 silently → Claude proceeds normally
16
```

Никаких фоновых процессов, никаких демонов, никаких файловых систем. Протоколы. Хук — это триггер, а собственная система фоновых задач Claude Code обеспечивает асинхронное выполнение.

Предварительные условия

- Установлен ключ API OpenRouter
- Claude Code (`claude` в PATH)
- Установлен Codex CLI (`codex` в PATH)
- Bun или среда выполнения Node.js

Настройка OpenRouter

Оба агента направляют логический вывод через OpenRouter, но используют разные интерфейсы API с разными базовыми URL.

Claude Code

Задайте эти переменные среды в своем профиле оболочки (`~/.zshrc` , `~/.bashrc`). Не используйте файл `.env` — Claude Code их не читает.

```
1 export ANTHROPIC_BASE_URL="https://openrouter.ai/api"
2 export ANTHROPIC_AUTH_TOKEN="sk-or-..."
3 export ANTHROPIC_API_KEY=""
```

⚠ Базовый URL-адрес: `https://openrouter.ai/api` — без `/v1` суффикса. Это скин OpenRouter для Anthropic, который использует собственный протокол Anthropic. Использование `/v1` приводит к ошибкам «модель не найдена». `ANTHROPIC_API_KEY` должно быть явно пустым, чтобы Claude Code не проходил аутентификацию напрямую через Anthropic.

Чтобы убедиться в этом, запустите `/status` в сеансе Claude Code. Подробнее см. в полном [руководстве по интеграции Claude Code](#).

Codex CLI

Создать или отредактировать `~/.codex/config.toml` :

```
1 [model_providers.openrouter]
2 name = "openrouter"
3 base_url = "https://openrouter.ai/api/v1"
4 env_key = "OPENROUTER_API_KEY"
```

Затем укажите ключ API:

```
1 export OPENROUTER_API_KEY="sk-or-..."
```

Во время выполнения передайте `-c 'model_provider="openrouter"'` для выбора провайдера OpenRouter.

⚠ Common pitfalls:

- Codex CLI does not have a `--provider` flag — use `-c` for all runtime config overrides
- The TOML section is `[model_providers.openrouter]`, not `[provider.openrouter]`
- Codex uses `https://openrouter.ai/api/v1` (with `/v1`), while Claude uses `https://openrouter.ai/api` (without `/v1`) — they use different protocol skins

Подробнее см. в полном [руководстве по интеграции Codex CLI](#).

Understanding the Stop Hook

Claude Code has a hook system configured in `settings.json`. The key hook for this use case is `Stop`, which fires every time Claude finishes a response cycle.

How decision: "block" works

1. The hook command runs and outputs JSON to stdout
2. If the JSON contains `"decision": "block"` with a `"reason"` string, Claude Code:
 - Does not stop — it continues the conversation
 - The `reason` text is injected into Claude's context as new information
 - Claude processes it and acts on it (e.g., spawning a background task as instructed)
3. If the command exits 0 with no JSON output, Claude proceeds normally (no blocking)
4. If the command exits non-zero, it's treated as a non-blocking error

Это ключевой механизм: команда `check` использует `decision: "block"` для внедрения в контекст Claude краткого описания статистики различий и инструкций по проверке. Claude видит краткое описание, решает, требуют ли изменения проверки, и, если да, запускает команду проверки с помощью своего инструмента Bash. Фоновая задача отображается в списке задач Claude — ее можно увидеть, отслеживать и завершить.

Settings file locations

In order of precedence:



- `~/.claude/settings.json` — global (all projects)
- `.claude/settings.json` — project (shareable, committed)
- `.claude/settings.local.json` — project (local, gitignored)

Используйте `.claude/settings.local.json` для хука проверки, чтобы он не влиял на других участников совместной работы.

Hook configuration

```
1 {
2   "hooks": {
3     "Stop": [
4       {
5         "hooks": [
6           {
7             "type": "command",
8             "command": "redline check",
9             "timeout": 10
10          }
11        ]
12      }
13    ]
14  }
15 }
```

Тайм-аут составляет 10 секунд — проверка выполняется быстро. Это намного меньше, чем 300 секунд, которые потребовались бы для синхронной проверки.

Why Async Matters

A simpler synchronous approach — running the full review inside the Stop hook — has three problems:

1. **Blocking.** `codex exec review` can take minutes. A synchronous Stop hook blocks Claude Code the entire time — no progress visibility, no way to cancel.
2. **No filtering.** The hook fires after every response, even when Claude just answered a question and made no code changes.

3. Duplicate reviews. Nothing prevents the hook from triggering a second review while one is already running.

Асинхронный подход решает все три задачи: проверка выполняется быстро (<1 секунды), срабатывает только при наличии изменений, хэширует статистику различий для пропуска, когда с момента последней проверки ничего не изменилось, а в тексте причины Клоду предлагается пропустить, если проверка уже запущена. Фактический просмотр выполняется как фоновая задача Claude Code, которую пользователь может отслеживать и завершать.

The Two-Command Architecture

The tool splits into two commands: `check` (the fast gate, called by the hook) and `review` (the actual work, spawned by Claude as a background task).

Building the check command

The check command runs on every Stop event and must complete in under a second.

```
1 import { execSync } from "child_process";
2 import {
3   existsSync,
4   readFileSync,
5   writeFileSync,
6 } from "fs";
7 import { join } from "path";
```

... See all 83 lines

Проверка использует `git diff --stat HEAD` для краткого описания того, что изменилось, и `git status --porcelain` для неотслеживаемых файлов. Она хэширует статистику различий и сохраняет ее в `.git/redline-last-diff` — если различия не изменились с момента последней проверки, хук завершает работу без вывода сообщений. Это предотвращает многократное срабатывание хука при одних и тех же различиях. Статистика различий включена в текст причины, чтобы Claude мог решить, требуют ли изменения проверки.

Building the review command



The review command is spawned by Claude as a background task. It streams Codex output in real-time for progress visibility, then prints a final summary.

```
1 import { spawn } from "child_process";
2 import { readFileSync, unlinkSync } from "fs";
3 import { join } from "path";
4 import { tmpdir } from "os";
5
6 async function review(model?: string): Promise<void> {
7   const outputFile = join(
8     tmpdir(),
9     `review-${model}.txt`
10  )
```

... See all 57 lines

Ключевые детали:

- `codex exec review --uncommitted` — reviews all staged, unstaged, and untracked changes
- `stdio: "inherit"` — streams Codex output in real-time so the background task shows progress
- `-o <file>` — writes the last agent message to a file for reliable output capture
- `-c 'model_provider="openrouter"'` — routes through OpenRouter
- Optional: `-c 'model="openai/gpt-5.4-pro"'` for model override
- Exit code is checked — if Codex fails and produced no output, the tool exits with an error

The reason text that instructs Claude

The check command's `reason` field includes the diff stat and lets Claude decide whether to review:



openrouter

Redline: Here is a summary of uncommitted changes

```
2  since the last review:
3
4  src/commands/check.ts | 25 ++++++-----
5  src/commands/review.ts | 12 +++++-----
6  2 files changed, 22 insertions(+), 15 deletions(-)
7
8  If these changes are substantial enough to warrant
9  a code review (e.g., new logic, bug fixes, refactors
10 – not just formatting or comments), run the
11 following command as a background task:
12
13   redline review
14
15 If the changes are trivial, or a review is already
16 running, skip it. When a review completes, assess
17 the findings and inform the user of any issues.
18
```

Клод просматривает краткое описание, оценивает, являются ли изменения существенными, и либо запускает проверку как фоновую задачу, либо пропускает ее. Когда проверка завершается, Клод читает потоковый вывод и представляет результаты.

Installing and Removing the Hook

The tool should provide commands to programmatically install and remove the hook from `.claude/settings.local.json`.

Install

```
1  redline install
```

Прочитать `.claude/settings.local.json`, выполнить глубокое слияние. Остановить запись в хуке и выполнить обратную запись. Создать `.claude/` каталог, если необходимо. Идемпотентность: если хук уже существует с той же конфигурацией,

пропустить. Если он существует с другой моделью, обновить его. Идентифицируйте свои хуки по префиксу команды (команды, начинающиеся с `"redline"`).

Полученный файл:

```
1  {
2    "hooks": {
3      "Stop": [
4        {
5          "hooks": [
6            {
7              "type": "command",
8              "command": "redline check",
9              "timeout": 10
10           }
11         ]
12       }
13     ]
14   }
15 }
```

Remove

```
1  redline off
```

Отфильтровать записи хуков, у которых `command` начинается с `"redline"` . Очистить пустые массивы и объекты (удалить `Stop: []` если пусто, удалить `hooks: {}` если пусто).

Implementation



openrouter

```
import {
2   readFileSync,
3   writeFileSync,
4   mkdirSync,
5 } from "fs";
6 import { join } from "path";
7
8 const SETTINGS_PATH = join(__dirname, "settings.json");
```

... See all 104 lines

Putting It Together

The full CLI has four commands:

```
1 # Install the hook
2 redline install
3
4 # Install with a specific review model
5 redline install --model openai/gpt-5.4-pro
6
7 # Remove the hook
8 redline off
9
10 # Run a review manually (prints to stdout)
11 redline review
12
13 # Fast gate check (called by the Stop hook)
14 redline check
```

Full CLI entry point



openrouter

```
const args = process.argv.slice(2);
2 const command = args[0];
3
4 const modelFlag = args.indexOf("--model");
5 const model =
6   modelFlag >= 0 ? args[modelFlag + 1] : undefined;
7
```

... See all 37 lines

Testing

Verify hook installation

```
1 redline install
2 cat .claude/settings.local.json
```

Вы должны увидеть запись Stop hook с командой `redline check` и 10-секундным таймаутом.

Test check with no changes

Ensure your working tree is clean, then:

```
1 redline check
```

Без вывода — проверка завершается без вывода, если нет незафиксированных изменений.

Test check with changes

Make a small change to any file, then:

```
1 redline check
```

Вы должны увидеть JSON с `"decision": "block"` и `"reason"` внутри, содержащим сводную статистику различий и инструкции по проверке.

Test review

```
1 redline review
```

Вы должны увидеть потоковую передачу данных из Codex, за которой последует краткое описание проверки.

Full integration test

1. Install the hook: `redline install`
2. Start Claude Code: `claude`
3. Ask Claude to make a small code change
4. When Claude finishes, the Stop hook fires the check — if there are uncommitted changes, Claude spawns the review as a background task
5. You can keep working while the review runs
6. When the review completes, Claude reads the output and presents any findings

Limitations and Future Work

Claude Code only

This pattern relies on Claude Code's `decision: "block"` hook output, which injects instructions directly into the agent's conversation. Codex CLI's hook system (as of early 2026) is more limited — its `Stop` hook is fire-and-forget and cannot feed structured output back into the model's context. Native hooks (`[[hooks]]` in `config.toml`) support several events, but only `SessionStart` can feed stdout into the model. There is no equivalent of `decision: "block" + reason` for injecting mid-session instructions. When Codex adds full structured hook output support, this pattern can be extended.

Review granularity

The review covers everything uncommitted — all staged, unstaged, and untracked changes. It does not distinguish between changes Claude just made and pre-existing uncommitted work. Future versions could use smarter change detection (e.g., diffing against a baseline snapshot taken before Claude's response).

Resources

- [Reference implementation \(redline\)](#)
- [Claude Code integration](#)
- [Codex CLI integration](#)
- [OpenRouter Activity Dashboard](#)